

Data Structures: Set Operations in Python

Solution Implementation

January 16, 2026

1 Overview

This document outlines a robust implementation of common set operations using Python's built-in set data structure. The implementation ensures $O(1)$ average time complexity for primary operations and handles edge cases such as removing non-existent elements.

2 Implementation Code

```
1 class Solution:
2     def insert(self, s, x):
3         """Adds an element x to the set s."""
4         s.add(x)
5         return s
6
7     def print_contents(self, s):
8         """Prints elements in sorted order."""
9         sorted_elements = sorted(s)
10        if not sorted_elements:
11            print("Set is empty")
12        else:
13            print(*(sorted_elements))
14        return sorted_elements
15
16    def erase(self, s, x):
17        """Safely removes x from the set."""
18        s.discard(x)
19        return s
20
21    def find(self, s, x):
22        """Returns 1 if x exists, otherwise -1."""
23        return 1 if x in s else -1
24
25    def size(self, s):
26        """Returns the count of unique elements."""
27        return len(s)
```

Listing 1: Python implementation of Set Operations

3 Methodology and Complexity

The following table summarizes the time complexity of the operations provided in the `Solution` class:

Operation	Description	Complexity (Average)
Insert	Adds an element to the set	$O(1)$
Erase	Removes an element (using discard)	$O(1)$
Find	Membership check	$O(1)$
Size	Returns length of set	$O(1)$
Print	Sorting and iteration	$O(N \log N)$

Table 1: Time Complexity Analysis

4 Key Features

- **Safety:** The use of `discard()` instead of `remove()` prevents the program from raising a `KeyError` if the element is not found.
- **Uniqueness:** Python sets internally handle duplicates, ensuring the `insert` operation maintains mathematical set properties.
- **Readability:** Logic is implemented using Pythonic shorthand for better performance and cleaner syntax.